

The Prompt Clinic

A team discussion exercise for Prompt Engineering and Task-to-Prompt Mapping

Format: Small teams (3 to 5). Discuss inside your team, capture answers, then report out to the full group.

Time: About 35 minutes of team time, then a group readout.

What you need: This handout, something to write on, and one person willing to defend your team's calls out loud.

The Setup

You are the newly formed AI platform team at **Cordwell Home & Hardware**, a fictional home-improvement retailer. The company wants to put an LLM behind its customer support desk: routing tickets, pulling structured data out of messy messages, and summarizing long threads for agents.

Product has been "prototyping with prompts" for a few weeks and just dropped a pile of prompts and design decisions on your desk. Some of it is solid. Some of it is quietly broken. Your job today is triage: diagnose what is wrong, decide what pattern fits, and design the parts that were never really designed.

You do not need to write working code today. You need to think like the team that has to run this in production on Monday.

How this maps to the decks: Round 1 is prompt anatomy (the four components). Round 2 is the pattern taxonomy (zero-shot, few-shot, chain-of-thought). Round 3 is classification design (label policy, boundary cases, tie-break, abstention). Everything you need was in Modules 01 and 02.

Suggested team time budget: R1 ~10 min, R2 ~10 min, R3 ~15 min. Keep moving. Partial answers you can defend beat perfect answers you never reach.

Round 1: Anatomy Triage

A teammate wrote this prompt to "figure out what support messages are about":

Look at this customer message and tell me what it's about:

"My online order #4471 shows delivered but the mulch never came, and now the app won't let me start a return."

It technically returns something every time. It also returns something different almost every time.

Discuss:

1. Module 01 says every production prompt has four components: instruction, context, constraints, and output contract. Which of the four are present, which are weak, and which are missing entirely here?
2. Name two concretely different outputs this prompt could produce for the exact same input. Why does that ambiguity exist?
3. Rewrite it into a production-grade prompt that includes all four components. Assume the downstream system needs to parse the result automatically.

Bring back: Your rewritten prompt, plus a one-line note on which single change you think buys the biggest jump in reliability.

Round 2: Pattern Pick

Module 01 gives you three core patterns: zero-shot, few-shot, and chain-of-thought. Below are three real tasks from the Cordwell backlog. For each, your team picks one pattern and defends it.

Task A. Route incoming tickets into five well-defined, everyday intents (order status, returns, billing, account access, product question). Volume is roughly 40,000 tickets a day.

Task B. Extract a structured record from each ticket: product name, issue type, and an internal urgency code from Cordwell's own five-level scale (P0_SAFETY down to P4_COSMETIC). The urgency scale is company-specific and appears nowhere in the model's training data.

Task C. Decide whether a warranty claim is covered. Coverage depends on purchase date, product category, whether the damage is "normal wear," and whether a receipt exists. The policy has several interacting conditions.

Discuss:

1. For each task, which pattern fits best, and what specifically about the task drove the choice?
2. One of these tasks is a trap: a pattern that "feels thorough" would add real cost and latency for little accuracy gain at 40,000 requests a day. Which task, which tempting-but-wrong pattern, and why is it wrong here?
3. Where a single pattern is not enough on its own, what would you combine it with?

Bring back: Your pattern choice for A, B, and C in one line each, and the identity of the trap task.

Round 3: Classification Design (the main event)

Product handed you this "label policy." It is barely a policy.

Labels: `technical_issue`, `billing_inquiry`, `account_access`, `feature_request`

Here is a sample of real-looking tickets it has to handle:

1. "The checkout page throws a 500 error when I apply my pro-rewards code."
2. "I was double charged for the same paint order last Tuesday."
3. "Can't log into my pro account to see my past invoices."
4. "Please add a dark mode to the mobile app."
5. "help"
6. "Do you ship to Canada?"
7. "My circular saw arrived cracked AND I want a full refund."
8. "This is NOT a defect, your install manual is just wrong."

Discuss and design:

1. **Tighten the labels.** Rewrite the four label definitions so a labeler could apply them consistently. Each definition should say what it includes and, just as importantly, what it excludes. (See the "strong label policy" slide in Module 02 for the shape of this.)
2. **Find the boundaries.** At least three tickets above are boundary cases that could plausibly land in more than one label, or in no label at all. Identify them and say why each is hard.
3. **Write a tie-break rule.** For tickets that legitimately match multiple labels, define a priority order and justify it in terms of business impact, not vibes. Then apply it: what does ticket 3 resolve to, and ticket 7?
4. **Handle the ones that do not fit.** Tickets 5 and 6 are not really any of the four labels. Design your abstention behavior: what new label or labels do you add, and what is the rule for when the model should use them instead of guessing?
5. **Define the contract.** Sketch the JSON your classifier should return for one ticket. What fields beyond the label itself would you want in production, and why?

Bring back: Your tie-break priority order, your resolution for tickets 3 and 7, and the JSON contract for one ticket of your choice.

Group Readout: Prompts Are Code

When we come back together, each team shares three things:

- **One fix** you would make first, from any round.
- **One pattern decision** your team actually argued about, and how it resolved.
- **One label boundary** that surprised you once you looked closely.

We will use those to pull out the throughline of both decks: a prompt is not a magic phrase, it is a component you version, test, validate, and measure like any other part of the system.

Fun rule

Bonus credit for the team that catches the Round 2 trap with the tightest cost reasoning, or surfaces a Round 3 boundary case no other team flagged, and can explain precisely why it is hard. Confidence without a reason does not count. That is sort of the whole point.